

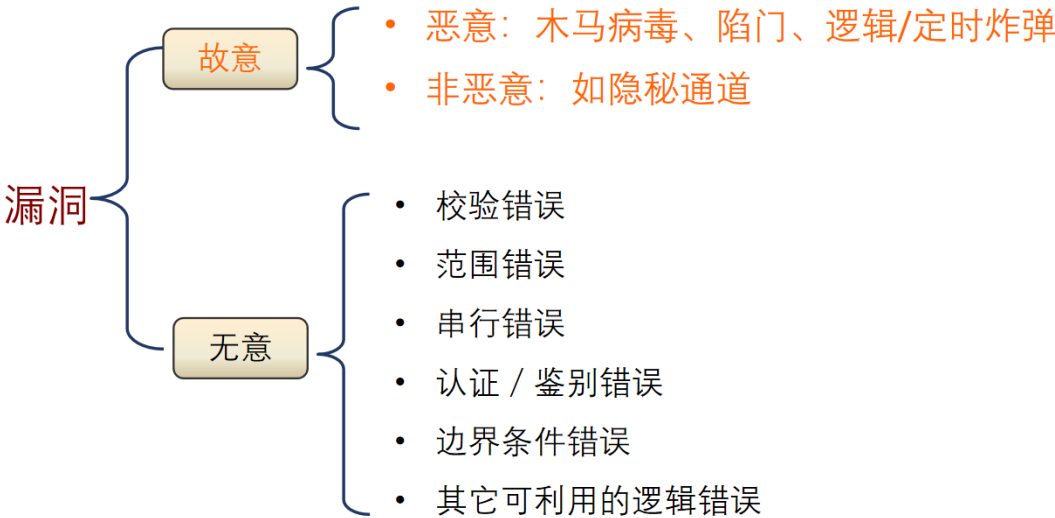
第七章 专项测试

7.2 安全性测试

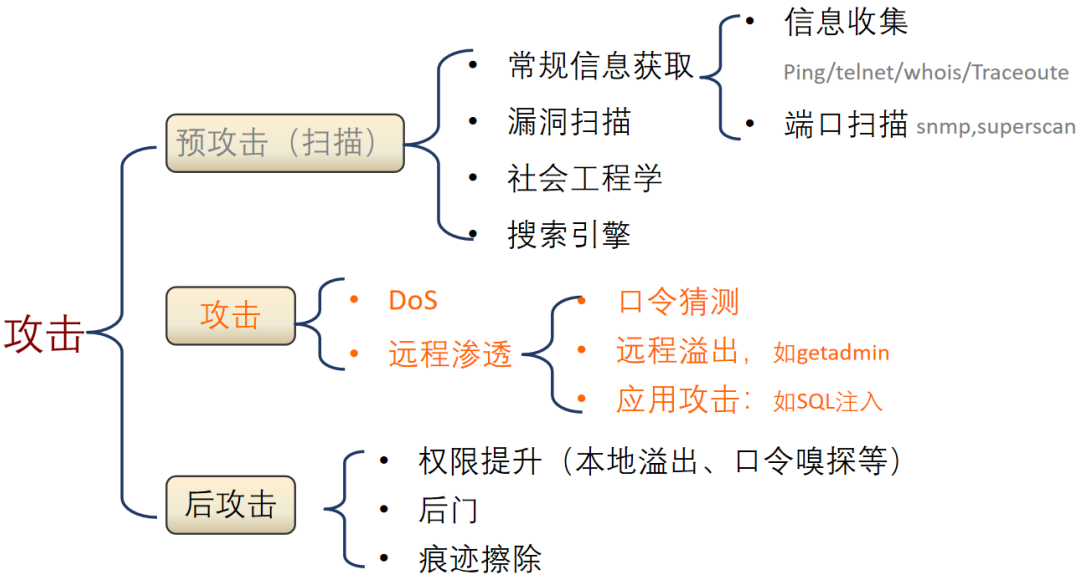
威胁模型与安全漏洞模式

漏洞

- 脆弱性（vulnerability），又称弱点或漏洞，是IT系统中存在的可能被威胁利用造成损害的薄弱环节，脆弱性一旦被威胁成功利用就可能违犯安全策略, 对IT系统（资产）造成损害。
- 漏洞的来源



危险、攻击手段



- 示例：ping命令。
 - 除了基本的连通性测试外，ping命令还可以用于收集其他有用的信息。例如，通过多次执行ping命令并观察结果的变化，用户可以评估网络的稳定

性；通过指定不同的参数和选项，ping命令还可以提供有关丢包率、抖动和路由等信息的详细输出，这些对于诊断和解决网络故障至关重要。

- 在使用ping命令时，需要注意一些安全和隐私方面的考虑。例如，攻击者可能会使用ping命令进行扫描和探测，以发现潜在的漏洞或目标主机。因此，对于重要的系统和网络，应该合理配置防火墙和安全策略，以限制或禁止未经授权的ping请求。
- 攻击带来的安全性问题
 - ❑信息泄露、破坏信息的完整性
 - ❑拒绝服务（DoS、DDoS）
 - ❑非法使用（非授权访问）、窃听
 - ❑业务数据流分析
 - ❑假冒、旁路控制
 - ❑授权侵犯（内部攻击）
 - ❑抵赖（来自用户的攻击）
 - ❑计算机病毒、恶意软件
 - ❑信息安全法律法规不完善
- 一些常见的安全漏洞模式
 - 缓冲区溢出漏洞
 - 攻击者写一个超过缓冲区长度的字符串，植入到缓冲区。
 - 这时可能会出现两种结果：一是过长的字符串覆盖了相邻的存储单元，引起程序运行失败，严重的可导致系统崩溃；另一个结果就是利用这种漏洞可以执行任意指令，甚至可以取得系统root特级权限。
 - 图像格式(WMF/TIFF/PNG)漏洞
 - 黑客可以在PNG文件中植入恶意程序，只要用户点击这些图片，就可能触发漏洞，进而引发远程程序攻击。
 - 并发漏洞
 - 通常发生在多个线程同时访问同一个共享资源，且没有进行锁操作或者同步操作的场景中。
 - 即并发安全问题。
 - SQL注入漏洞
 - 攻击者利用这个漏洞，可以将恶意输入的SQL命令插入到Web表单提交、输入域名或页面请求的查询字符串中，从而欺骗服务器执行恶意的SQL命令。
 - 这种攻击方式利用了现有应用程序的漏洞，使得攻击者能够执行后台数据库引擎的恶意SQL命令。
 - 造成SQL注入漏洞的原因主要有两个：一是对输入的数据没有进行充分的过滤，二是对发送到数据库的数据没有进行转义。这使得攻击者有可能通过精

心构造的输入来干扰或改变原有的SQL语句，从而达到执行恶意操作的目的。

- 跨站脚本攻击（XSS）

- 是一种网站应用程序的安全漏洞攻击，是代码注入的一种形式。
- 它允许恶意用户将代码注入到网页上，当其他用户在观看网页时就会受到影响。
- 这种攻击利用了网站对用户输入的不正确处理，使得恶意用户能够向受害者的网页中注入恶意脚本，这些脚本在用户浏览器中执行，从而导致安全风险。

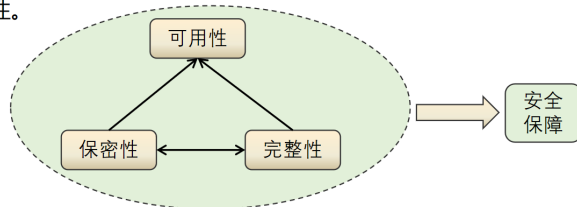
- 跨站请求伪造攻击（CSRF）

- 通常缩写为 CSRF 或者 XSRF，是一种挟制用户在当前已登录的Web应用程序上执行非本意的操作的攻击方法。
- 攻击者不需要控制放置恶意网址的网站，也不需要用户点击任何链接或按钮，只要用户在已登录的状态下访问了恶意网址，就可能触发攻击。
- 与跨网站脚本（XSS）相比，XSS利用的是用户对指定网站的信任，而CSRF则利用的是网站对用户网页浏览器的信任。攻击者通过一些技术手段欺骗用户的浏览器去访问一个用户曾经认证过的网站并运行一些操作（如发邮件、发消息，甚至进行财产操作如转账和购买商品）。由于浏览器曾经认证过，所以被访问的网站会认为是真正的用户操作而去执行。

- 安全性测试的范围与方法

- 安全目标

- **安全目标**是指能够满足一个组织或者个人的所有安全需求，通常强调CIA三元组的目标，即**保密性**(Confidentiality)、**完整性**(Integrity)和**可用性**(Availability)
 - 由于这些目标常常是互相矛盾的，因此需要在这些目标中达到最佳平衡。例如，简单地阻止所有人访问一个资源，就可以实现该资源的保密性，但这样就不满足可用性。



- 安全性要求

- **保密性**：确保信息只被授权人访问，信息即使被截取也不能了解信息的真实含义
- **完整性**：保护信息和信息处理方法的准确性和原始性，包括数据的一致性，防止数据被非法用户篡改
- **可用性**：确保授权的用户在需要时可以访问信息
- **真实性**：保证信息来源真实可靠
- **不可抵赖性**：用户对其行为不可否定
- **可追溯性 (Accountability)**：确保实体的行动可被跟踪
- **可控制性**：对信息的传播及内容具有控制能力
- **可审查性**：对出现的网络安全问题提供调查的依据和手段

- 软件安全性需求

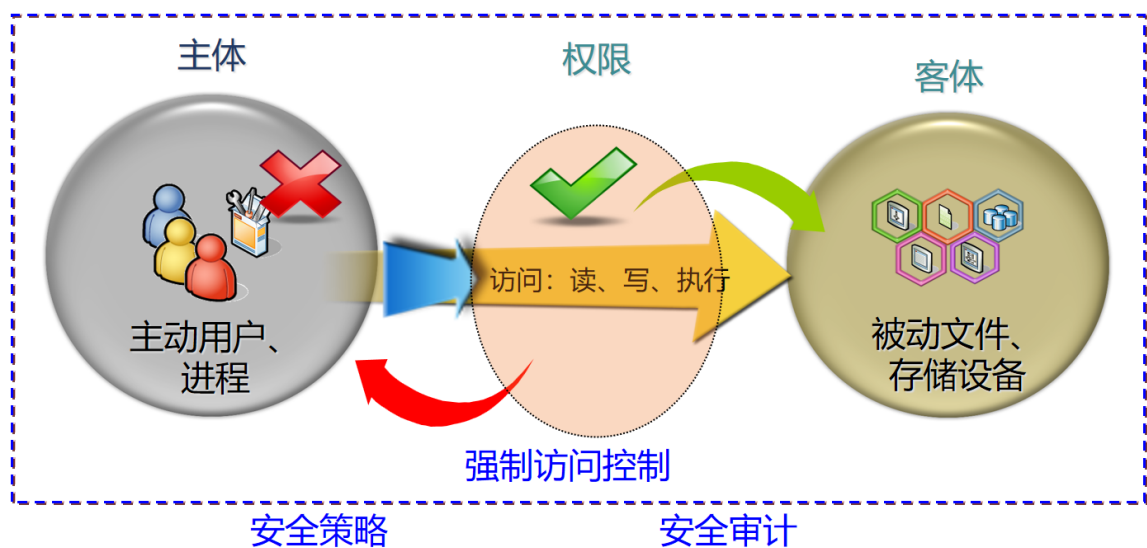
系统安全性

- 用户身份认证
- 用户权限限制
- 异常处理
- 入口验证机制
- 系统强壮性设计

数据安全性

- 加密
- 数据结构
- 安全存储
- 安全访问
- 认证
- 备份

- 系统安全的基本机制



- 安全三大保障机制

- 预防
- 恢复
- 支撑

- 完整的安全策略示例

- 可接受的加密控制
- 可接受的使用控制
- 模拟线路和拨入访问控制
- 防病毒流程
- 应用程序提供上（ASP）控制
- 引入评估控制
- 审核和风险评估
- 自动转发电子邮件控制
- 数据库信任编码

- 外部网络访问控制
- 远程访问和VPN安全控制
- 敏感信息控制
- Internet DNS设备控制
- 口令保护
- 路由器安全管理
- 服务器安全管理
- 第三方网络连接协议
- 无线通信控制

- 安全性测试的目标

- 通常的目标

- （强制）访问控制
 - 身份鉴别
 - 完整性保护（数据存储、传输和处理）

- 进一步的目标

- 隐蔽通道分析
 - 可信路径
 - 加密算法、代码规范
 - 潜在的安全漏洞

数据的机密性、完整性、可用性

- 软件安全性测试的范围

- 安全功能测试 (Security Functional Testing): 数据机密性、完整性、可用性、不可否认性、身份认证、授权、访问控制、审计跟踪、委托、隐私保护、安全管理等

- 安全漏洞测试 (Security Vulnerability Testing): 从攻击者的角度, 以发现软件的安全漏洞为目的。安全漏洞是指系统在设计、实现、操作、管理上存在的可被利用的缺陷或弱点

- 功能性测试 vs. 安全性测试

- ❑ **功能性测试**：软件做它应该做的事，验证正确的输出

发现不正确的输出 / 行为 / 缺陷(Bug)

- ❑ **安全性测试**：软件不做它不应该做的事，应用输入的验证，以避免不安全的事情发生。在测试软件系统中对危险防止和危险处理设施进行测试，以验证其是否有效

- 安全性测试的任务

- ❑ 全面检验软件在**需求**规格说明中规定的防止危险状态措施的有效性和在每一个危险状态下的反应
- ❑ 对软件**设计**中用于提高安全性的结构、算法、容错、冗余、中断处理等方案，进行针对性测试
- ❑ 在**异常**条件下测试软件，以表明不会因可能的单个或多个输入错误而导致不安全状态
- ❑ 对安全性**关键的软件单元、组件**，单独进行加强的测试，以确认其满足安全性需求

- 安全等级

- | | |
|------|------------------------|
| I. | 用户自主保护级
控制资源访问 |
| II. | 系统审计保护级
审计访问行为 |
| III. | 安全标记保护级
实现强制访问控制 |
| IV. | 结构化保护级
可信计算基础结构化 |
| V. | 访问验证保护级
验证所有过程、可信恢复 |

- 安全性测试方法

- ❑ **基于威胁的方法**

从软件**外部**考察其安全性，识别软件面临的安全威胁并测试其是否能够发生

- ❑ **基于漏洞的方法**

从软件**内部**考虑其安全性，识别软件的安全漏洞，如借助特定的漏洞扫描工具

静态测试方法

- ❑ 工具扫描/分析代码
- ❑ 人工评审：架构/代码
- ❑ 专家安全风险评估

动态测试方法

- ❑ 功能测试方法
- ❑ 模糊测试方法
- ❑ 渗透测试方法

- 安全性功能测试

❑ 身份认证测试

(如DCE/Kerberos、基于公共密钥等认证机制)

❑ 访问控制策略验证

(如入口访问、权限、目录等控制)

示例：口令

❑ 规则验证

❑ 负面测试

❑ 传输检查

❑ 存储检查

❑ 控件兼容性

❑

● 模糊测试方法

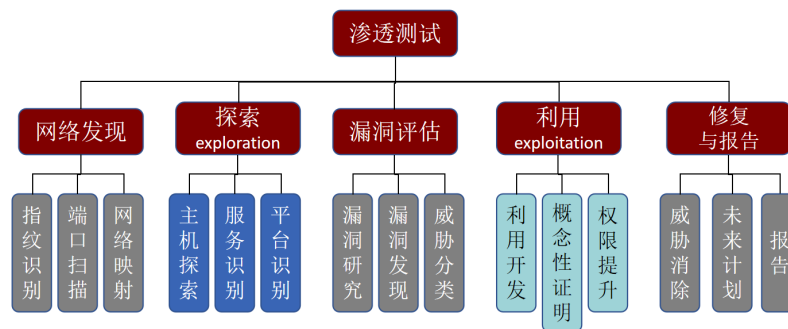
- 模糊测试：在一个被测试程序中附加上随机数据（fuzz）作为程序的输入。如果被测试程序出现问题（例如 Crash ,或者异常退出），就可以定位程序的缺陷。
- 是一种通过向目标系统提供非预期的输入并监视异常结果来发现软件漏洞的方法。
- 基本原理:通过向目标软件系统提供大量随机或异常的输入数据，模拟真实环境中各种可能的情况，以触发异常响应，如崩溃、异常行为或错误，从而发现潜在的安全问题。
- 模糊测试的巨大优势：测试设计极其简单，系统的行为先入为主。

几种不同模糊器的构造

- ❑ 黑盒随机模糊，对正确格式的输入数据进行随机变异，然后用这些变异的输入运行程序，看是否能够触发异常。
- ❑ 基于语法的模糊，是模糊复杂格式输入的替代方法，需要指定输入格式的输入语法、哪些输入部分要进行模糊化以及如何模糊化。
- ❑ 白盒模糊处理，由微软研究院于2008年首创，这种方法包括：动态地执行被测程序，从执行过程中遇到的条件分支收集输入约束。然后，系统地逐个否定所有这些约束，并使用约束求解器求解，其解被映射到执行不同程序执行路径的新输入。使用系统搜索技术重复这个过程，试图扫描程序的所有可行的执行路径。

● 渗透测试

- 采用探索式测试方式，模拟黑客可能使用的攻击技术和漏洞发现技术，对目标系统的安全做深入的探测，发现系统最脆弱的环节，直观地发现问题。



- 实施策略

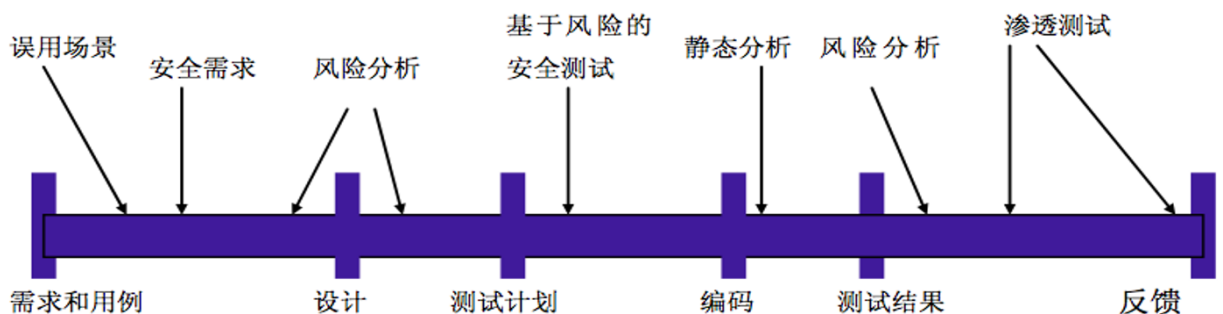
- ▣ **全程监控**：采用类似wireshark的嗅探软件进行全程抓包嗅探
- ▣ **择要监控**：对扫描过程不进行录制，仅仅在数据分析后，准备发起渗透前才开启软件进行嗅探
- ▣ **主机监控**：仅监控受测主机的存活状态
- ▣ **指定攻击源**：多方监控指定源（某主机）进程、网络连接、数据传输等
- ▣ **对关键系统**，可以采用对目标的副本进行渗透测试

- 贯穿研发生命周期的安全性测试

- 软件安全实施方针

- ▣ **软件安全是一种系统级的问题**，需要考虑**安全机制**（如访问控制）和**基于安全的设计**（如使攻击难以实施的健壮设计）
 - ▣ 软件安全必须成为完整的**软件开发生命周期方法**的一部分

- 将风险分析、安全测试与软件开发过程结合起来，在软件开发的各个阶段进行误用模式、异常场景、风险分析以及渗透测试等，尽早地发现高风险的安全漏洞。



- 体系结构风险分析

- ▣ **工件**：设计和说明书
 - ▣ **发现风险的例子**：对关键数据的区分和保护很糟糕；Web服务未能验证调用代码及其用户
 - ▣ **系统体系**必须连贯一致，并提供统一的安全防线，应用文档清晰地记录各种前提假设，并确定可能的攻击。
 - ▣ **体系结构风险分析**都是必需的，以揭示体系结构瑕疵，并进行评估、降低风险，否则后果严重。在软件生命周期的各个阶段都可能出现风险，因此，应采用持续的风险管理方法，并不断地监控风险

- 基于风险的安全测试

- **工件：单元和系统**

- **安全测试策略：**

- ① 用标准功能测试技术来进行的安全功能性测试；
- ② 以攻击模式、风险分析结果和滥用案例为基础的基于风险的安全测试。

- **标准的质量保障方法**可能不能揭示所有严重的安全问题。

- **安全测试**是为了保证坏的事情不会发生，像攻击者一样地考虑问题很重要

- Web安全性测试

- 渗透测试点

- 检查应用系统架构,防止用户绕过系统直接修改数据库
 - 检查身份认证模块,用以防止非法用户绕过身份认证
 - 检查数据库接口模块,用以防止用户获取系统权限
 - 检查文件接口模块,防止用户获取系统文件
 - 检查其他安全威胁

- 渗透测试

- 注入攻击 (Injection)

- SQL注入式攻击
 - XML注入

- 跨站点脚本攻击 (XSS)

- XSS: 允许恶意用户将代码植入到“供其它用户使用的web页面”中

- 例如: 一般输入域中能输入:

- `javascript:alert(document.cookie)`

- 在其它页面显示,具有隐藏性

- 受害者的敏感数据、session、cookie等会泄漏

- 更严重的是黑客能监控用户的行为、用户被迫访问其它站点

- 跨站点请求伪造(CSRF)

- 未能限制URL访问

- 会话劫持

- 通过Cookie或者Session判断和跟踪不同的用户,由于Cookie失效时间很长,攻击手段一般是盗取用户Cookie然后伪造Cookie冒充该用户,从而劫持会话。

- 无效认证和Session管理方式

- 用户密码在数据库中用明文保存、用户登录使用未加密连接
- 用户密码可以被特殊操作覆盖。例如不正确的实现密码更改功能、恢复密码功能等都有可能造成密码被直接更改。
- 网页URL中包含Session ID。
- Session ID不会timeout，或者session/token/SSO token在登出的时候未失效。
- 用户认证信息、Session ID使用未加密连接传输。

- 未验证跳转

- 有些页面跳转会根据用户输入来决定，这就给了攻击者可乘之机，从而将用户导向恶意网站或者未授权链接。
- 防御
 - 避免跳转
 - 不要根据用户输入来跳转
 - 如果必须根据输入跳转，验证该输入并且该用户具备访问该目标路径的权限
 - 如果必须根据输入跳转，推荐根据用户输入来内部决定对应的跳转目标，不直接使用输入

- 功能级权限控制缺失

- 权限控制一般是写在代码中或者通过程序的配置文件来完成，但开发者人员容易忘记添加一些功能的权限控制代码。
- 防御
 - 不要hard code权限控制，需要建立一种可以比较容易更新和监测权限控制的机制
 - Hard code（硬编码）是指在程序代码中直接嵌入固定的值或逻辑，而不是通过配置文件、数据库、或用户输入等动态方式来获取这些值或逻辑。
 - 默认拒绝所有访问，访问任何功能都需要被赋予特定的权限
 - 如果某功能在一个workflow中，需要确认所有的前提条件都在正确的状态，然后允许访问该功能

- 7.3 兼容性测试

- 软件与硬件兼容性问题
- 软件系统之间的兼容性
 - 最常见的数据兼容性问题
 - 文件打不开
 - 数据显示不对
 - 系统崩溃
- 相同系统之间的数据兼容
 - 客户端版本升级

- 服务器端版本升级
- 不同系统之间的数据兼容
- 软件环境(组合)兼容性测试

1. Pairwise组合测试、正交实验法

2. 根据市场调查数据，设计组合矩阵表

	PC				移动设备			
	Win 10	Win 11	Linux	OS X	鸿蒙	iOS 16	Android	WP10
IE 10	√							√
IE 11	√	√						√
Edge	√	√						√
Firefox	√		√	√	√	√	√	
Chrome		√	√	√		√	√	
Safari				√				√
...					BB10			

• 7.4 可靠性测试

• 7.4.1 可靠性测试方法

可靠性：在规定的一段时间和条件下，软件能维持其性能水平的能力有关的一组属性，可用成熟性、容错性、易恢复性三个基本子特性来度量。

- 成熟性度量：通过错误发现率DDP（Defect Detection Percentage）来表现。
DDP越小，软件越成熟。

$$\text{DDP} = \frac{\text{测试发现的错误数量}}{\text{已知的全部错误数量}}$$

- 容错性测试在下面一节介绍
- 恢复性的测试先设法（模拟）使系统崩溃、失效等，然后计算其系统和数据恢复的时间来做出易恢复性评估。
- $\text{DDP} = \frac{\text{测试发现的错误数量}}{\text{已知的全部错误数量}}$ 。其中，已知的全部错误数量是测试已发现的错误数量加上可能会发现的错误数量之和。

• 7.4.2 容错性测试

- **容错测试**是一种对抗性的测试过程。在这种测试中，通过各种手段让软件强制性地发生故障，或将把应用程序或系统置于（模拟的）异常条件下，以产生故障，例如设备输入/输出（I/O）故障或无效的数据库指针和关键字等

• 7.4.3 数据库并发控制测试

- **数据库的并发控制能力**是指在处理多个用户在同一时间内对相同数据进行同时访问的能力。
- 并发控制过冲突中，主要包括三类：
 1. 丢失数据
 2. 不可重复读数据
 3. 读“脏”数据

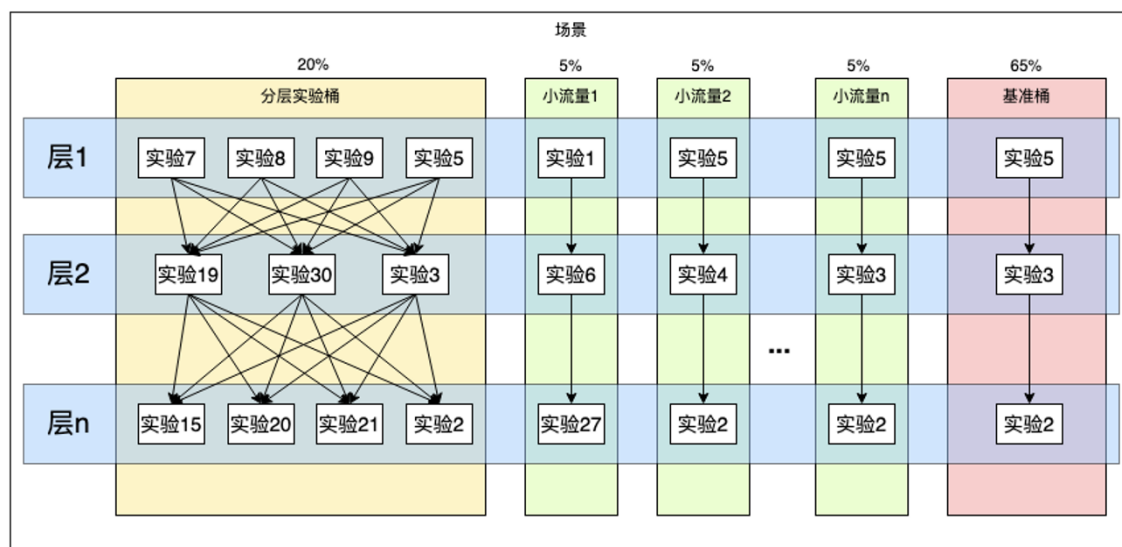
- **7.5 易用性测试：A/B测试**

- 良好的UI元素
 - 好的用户界面包括7个要素：符合标准和规范、直观性、一致性、灵活性、舒适性、正确性、实用性。
- 用户体验测试方法
 - **探索式测试**：可确定新产品应包含哪些内容和功能，可评估初步设计或原型的有效性和易用性。
 - **评估性测试**：在发布前或发布后对最新版本的测试，以确保用户直观使用并提供良好的用户体验。
 - **比较性测试**：比较两种或更多种产品或设计的易用性，并区分各自的优缺点，以确定哪种设计能提供最佳的用户操作体验。
- A/B测试
 - A/B测试（ABTest）是将用户分成不同的组，同时在线试验产品的不同版本，通过用户反馈的真实数据来确定哪一个方案更好。
 - 特点
 - **先验性**：采用流量分割与小流量测试的方式，先让线上部分小流量用户使用以验证设计，再根据数据反馈来推广到全流量，减少产品损失
 - **并行性**：同时运行两个或两个以上版本的试验完成对比分析，而且保证每个版本所处的环境一致的，避免流程周期长的问题，节省验证时间
 - **科学性**：基于统计的数据来做出决策，避免主观或经验的错误决策
 - ABTest适应的场景
 - **灰度发布**：技术&算法迭代
 - **功能优化**：界面模块、样式风格、交互方式等
 - **内容优化**：推广海报、落地页、内容模块、文案等
 - **运营优化**：运营策略、沟通话术等

- 核心概念

- **场景**：对应业务场景，场景之间完全独立
- **桶**：圈定了特定的用户流量，即不同的桶之间流量是互斥的。它从属于场景，一个场景可以有多个桶
- **层**：一类实验的集合，从属于场景，一个场景可以有多个层，处于同一层的各实验之间流量互斥，处于不同层的各实验之间流量正交，各实验流量之和为总流量
- **实验**：用来验证某个决定请求处理方式的功能或策略的一部分流量，通常用来验证某个功能或策略对系统指标（如PV/UV，CRT，下单转化率等）的影响
- **流量**：指所有用户请求
- **流量正交**：不同层之间流量分配方式完全独立，不会互相影响，符合MECE

A/B测试：桶、层、流量



- A/B测试流程

- 产生实验想法
- 评估实验优先级
- 设计和开发实验
- 分析数据
- 应用结果

- A/B测试SDK

- ① **实验配置动态分发**：实现针对场景标识+用户分流标识的一致性哈希算法，根据场景的实验流量配置，进行实验配置的动态分发
- ② **上报ABTest数据**：请求日志、业务自定义日志以及监控日志
- ③ **基于ABTest埋点规范生成前端埋点标识**：abTraceld（请求唯一标识）和bcm（请求结果标识）、事件标识（曝光view、点击、收藏、转发等）等追踪标识，用以透传到前端埋点来追踪用户行为

- 平台功能

- **ABTest元数据管理**，包括应用、场景与实验的创建、编辑，以及配置的发布等。
- **ABTest接入的开发和测试支持**。用户可以方便地查看完整可用的接入示例代码，可以输入分流用户标识进行测试，以及查询实时日志等。
- **实时报表和离线效果报表**。实时报表包含实时请求、点击(率)、转化率、千次曝光转化等相关指标数据，及其对比分析
- **异常监控和告警**。平台实时监控 SDK 上报的 ABTest 请求数、失败数和延迟时间等数据，一旦发现异常即发出告警

- 小结

本节小结

- 兼容性测试覆盖软件于硬件、软件与软件、数据之间的兼容性，而且数据兼容更为重要，因为数据更具价值
- 全链路压测可看作是性能测试的右移，全面地提升系统的稳定性。
- A/B测试是为了提高用户体验采用的科学方法，让数据说话
- A/B测试包括实验设计、流量分发控制算法等关键内容
- 建立一个灵活支持A/B测试的平台